

Konwersja z adresu liniowego na rzeczywisty

Zakładamy 64 bitowy tryb pracy procesora. Bezpośrednio po włączeniu procesor x86_64 znajduje się w trybie 32-bitowym z segmentacją, ale bez stronicowania. Należy przygotować tablice translacji i dokonać przełączenia przez rejestr CR0 (bit 31, PG).

```
mov eax, cr0
or  eax, 80000000h
mov cr0, eax
```

Sprawdźmy działanie translacji adresu na przykładzie adresu globalnej tablicy deskryptorów (GDT). W rejestrze GDTR znajduje się m.in. adres **liniowy** lokacji tablicy GDT. Trzeba pamiętać, że rejestr GDTR jest 48-bitowy/80-bitowy (dla trybu 32/64-bitowego), z czego młodsza część jest zawsze 16 bitowa, zaś najstarsze bity (odpowiednio 32/64) stanowią adres **liniowy** położenia tablicy GDT w pamięci.

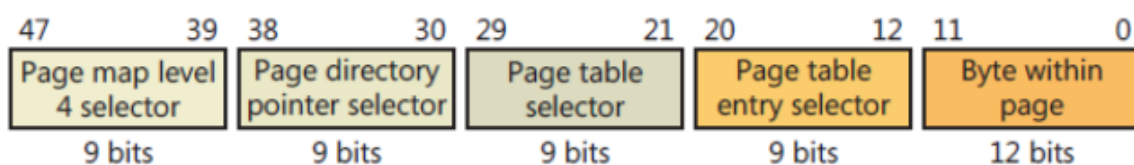
Prześledźmy to na przykładzie. W Windows 10 (po użyciu komendy `rM 0x100` w windbg), wartość rejestru wynosi:

```
gdtr=0xFFFFF80000b95000 gdtl=007F
```

Oznacza to adres tablicy jest równy 0xFFFFF800'00b95000, zaś rozmiar równa się 0x7F (dokładnie 0x80, gdyż jest to adres ostatniego bajtu, który należy do GDT) bajtów. Warto zwrócić uwagę, że debugger automatycznie rozdziela faktyczną zawartość rejestru GDTR na adres i rozmiar (nie ma takiego rejestru jak GDTL). Wyświetlenie tablicy GDT możliwe jest przy użyciu polecenia `dg <adres_początkowy_deskryptora> <nr_końcowy_deskryptora>`, czyli `dg 0 7F`.

W jaki sposób wyznaczyć położenie GDT w pamięci fizycznej?

Po pierwsze zgodnie ze schematem dzielimy adres liniowy GDT (0xFFFFF80000b95000) na pięć bloków (odpowiednio 9-9-9-9-12 bitowych, łącznie 48 bitów) zgodnie z poniższym schematem.



Przy naszym adresie daje to wartości:

1111 1111 1111 1111 | 1111 1000 0 | 000 0000 00 | 00 0000 101 | 1 1001 0101 | 0000 0000 0000

Daje to nam PML4E index=0x1F0, PDPE index=0x0, PDE index=0x5,

PTE index=0x195 i offset = 0

Po wyznaczeniu indeksów przystępujemy do faktycznego wyznaczenia położenia odpowiednich wpisów w tablicach.

Zaczynamy od rejestru CR3 (polecenie `r cr3`)

Przykładowo otrzymujemy: `cr3=0000 0000 50ac 6000`

Rejestr jest 64-bitowy, z czego 12 najmłodszych bitów określa pewne atrybuty, zaś 52 najstarsze bity adres fizyczny. Debugger podobnie jak w przypadku GDTR, dokonał już konwersji CR3 (wcięcie najstarszych bitów oraz przemnożenie przez 2^{12}).

Adres ten wynosi zatem `0x0000000050ac6000` i wskazuje na położenie początku tablicy PML4.

Chcąc uzyskać wpis musimy się przesunąć o `0x1F0` * 8 bajtów (mnożymy przez 8, bo każdy wpis w tablicy jest 64 bitowy) co daje `0xF80` bajtów przesunięcia. Interesuje nas więc adres fizyczny `0x0000050ac6F80`

Odczytujemy go:

```
kd> !dq 50ac6F80 L1
#50ac6f80 00000000`00199063
```

64-bitowa liczba `0x0000000000199063` to wartość PML4 Entry, z którego 52 najstarsze bity to adres bazowy (fizyczny) początku tablicy PDP, który rozszerzamy do 64 mnożąc przez 2^{12} i mamy adres fizyczny kolejnej tablicy, PDP = `0x00000000199000`.

Odczytany indeks wpisu dla tej tablicy to `0`, stąd odczytujemy bajty bez modyfikacji

```
0: kd> !dq 000199000 L1
# 199000 00000000`00198063
```

64-bitowa liczba `0x0000000000198063` PDP Entry, z którego 52 najstarsze bity to adres adres bazowy, który rozszerzamy do 64 bitów mnożąc przez 2^{12} i dodając `0x05` * 8 bajtów = `0x28` (bo `0x5` to index wyznaczony z adresu liniowego) mamy adres fizyczny kolejnej tablicy, PDT = `0x00000000198028`

```
0: kd> !dq 0000198028 L1
# 198028 00000000`0019a063
```

Zbliżyliśmy się ku końcowi. Wyznaczona wartość to adres ostatniej tablicy PT = `0x00019A000` (po przemnożeniu przez 2^{12}). Wyznaczony indeks = `0x195` * 8 = `0xCA8`, czyli adres PTE to `0x00019ACA8`.

```
0: kd> !dq 00019ACA8 L1
# 19aca8 00000000`00b95163
```

Zatem adres bazowy naszej strony zawierającej tablicę GDT to `0x00000b95000` (po wydobyciu i rozszerzeniu). Z adresu liniowego mamy offset = `0`, czyli na początku tej strony powinien być dummy descriptor.

Sprawdźmy

```
0: kd> !dq 0000b95000
# b95000 00000000`00000000 00000000`00000000
# b95010 00209b00`00000000 00cf9300`0000ffff
# b95020 00cffb00`0000ffff 00cff300`0000ffff
# b95030 0020fb00`00000000 00000000`00000000
# b95040 00008bb9`60800067 00000000`fffff800
# b95050 ff40f3fb`00003c00 00000000`00000000
# b95060 00cf9a00`0000ffff 00000000`00000000
# b95070 00000000`00000000 00000000`00000000
```

Jeśli wykonany tę operację na adresach liniowych otrzymamy

```
0: kd> dq @gdtr
```

```
fffff800`00b95000  00000000`00000000  00000000`00000000
fffff800`00b95010  00209b00`00000000  00cf9300`0000ffff
fffff800`00b95020  00cffb00`0000ffff  00cff300`0000ffff
fffff800`00b95030  0020fb00`00000000  00000000`00000000
fffff800`00b95040  00008bb9`60800067  00000000`fffff800
fffff800`00b95050  ff40f3fb`00003c00  00000000`00000000
fffff800`00b95060  00cf9a00`0000ffff  00000000`00000000
fffff800`00b95070  00000000`00000000  00000000`00000000
```

Widać, że bloki te mają identyczną zawartość

Dodatkowo możemy sprawdzić kroki pośrednie z wynikami uzyskanymi poprzez komendę konwersji adresów liniowych na fizyczne.

```
0: kd> !pte @gdtr
```

```
VA fffff80000b95000
PXE at FFFFF6FB7DBEDF80  PPE at FFFFF6FB7DBF0000  PDE at FFFFF6FB7E000028  PTE at FFFFF6FC00005CA8
contains 0000000000199063 contains 0000000000198063 contains 000000000019A063
contains 0000000000B95163
pfn 199 ---DA--KWEV pfn 198 ---DA--KWEV pfn 19a ---DA--KWEV pfn b95 -G-
DA-KWEV
```

Całość obrazuje poniższy rysunek.

GDTR FFFF80000b95000

CR3 0000 0000 50AC 6000

PML4	1000	50AC6	F80 = 4*1F0		
				00000000'00199063	0000000050AC6F80
				00000000'00000000	0000000050AC6000
GDT	400=0x80*8	B95		Deskryptor 1	
				Deskryptor 0	
Page Table	1000	19A	CA8=195*8	00000000'00B95163	0000000000019ACA8
				00000000'00000000	0000000000019A000
Page Direct. Pointer	1000	199		00000000'00198063	00000000000199000
Page Direct.	1000	198	28=5*8	00000000'0019A063	00000000000198028
				00000000'00000000	00000000000198000
	1000	1			
	1000	0			
Nazwa	Rozmiar	Numer ramki PFN	Offset	Adres fizyczny	

GDT i wznacznie adresu

liczba

w rejestrze GDT